T9 - Checklist Operation Cards (27 small drills)

One card per bullet of the instructor's practical-assessment checklist (Pandas, Matplotlib, SQLite, PyTorch). Learn the OPERATIONS, not one big solution - every card is tested and runs on its own.

How to use (20 minutes a day)

The instructor: "*Do not focus on memorising one complete solution. Instead, practise the individual operations and understand what each line of code is doing.*" So drill one operation at a time.

1. **Pick 4-5 cards** you have not ticked yet (about 4 minutes each). Start with the unticked rows of the tracker below.
2. **Cover the code box** with a sheet of paper. Read only the title, the Analogy and the Self-test question.
3. **Write the code from memory** on paper or in an empty `.py` file. Say what each line does out loud.
4. **Run it** (or run the card's function: `card_NN_...()` in `Practice/Solutions_py/practical_operation_drills.py`). Compare with the card. Use the "Check it worked" line to PROVE it worked.
5. **Tick the box** in the tracker only when you got it right from memory with no peeking. Next day, redo the ticked cards of two days ago.

Run all 27 drills at once (about 5 seconds, prints PASS/FAIL per card): `MPLBACKEND=Agg python practical_operation_drills.py` (without `MPLBACKEND=Agg` each plot card opens a window - close it to continue).

The tiny data the cards use (the drills file builds it for itself in a temporary folder):

```
sales.csv  (6 rows, 4 columns; the two "Book" rows are an exact duplicate)
Date,Product,Quantity,Price
2023-01-05,Pen,10,1.5          products.csv  (Cap is missing, Ink is extra)
2023-01-06,Book,2,12.0         Product,Category
2023-01-06,Book,2,12.0         Pen,Stationery   Book,Stationery
2023-02-01,Bag,1,30.0          Bag,Accessories  Ink,Stationery
2023-02-03,Pen,5,1.5
2023-03-09,Cap,3,8.0
```

Cards 15-19 assume `conn = sqlite3.connect("shop.db")` and `cur = conn.cursor()` are open (card 13) and table `sales` holds `sales.csv` (card 14). Cards 24-27 assume `X, y` from card 23, `model = nn.Linear(3, 1)` (or the `Net` of card 21), and `loss_fn`, `optimizer` from card 25. Every card assumes `import pandas as pd` and `import matplotlib.pyplot as plt` at the top.

Tracker

Tick `[x]` only when you can write the card from memory, run it, and explain every line.

[ ] Can do from memory	Card #	Checklist bullet	Where to read more
[ ]	1	Pandas: loading CSV files	T5_Pandas_NumPy, T3_Files_CSV_Exceptions
[ ]	2	Pandas: checking the shape of a DataFrame	T5_Pandas_NumPy
[ ]	3	Pandas: identifying and removing duplicates (identify)	T5_Pandas_NumPy
[ ]	4	Pandas: identifying and removing duplicates (remove)	T5_Pandas_NumPy
[ ]	5	Pandas: merging DataFrames using a common column	T5_Pandas_NumPy
[ ]	6	Pandas: selecting and filtering data	T5_Pandas_NumPy
[ ]	7	Pandas: saving a DataFrame to a CSV file	T5_Pandas_NumPy, T3_Files_CSV_Exceptions

[ ] Can do from memory	Card #	Checklist bullet	Where to read more
[ ]	8	Matplotlib: simple plots - scatter plot (+ title and axis labels)	T6_Matplotlib
[ ]	9	Matplotlib: simple plots - line plot (+ title and axis labels)	T6_Matplotlib
[ ]	10	Matplotlib: simple plots - bar chart (+ title and axis labels)	T6_Matplotlib, T5_Pandas_NumPy
[ ]	11	Matplotlib: simple plots - box plot (+ title and axis labels)	T6_Matplotlib
[ ]	12	Matplotlib: adding appropriate titles and axis labels	T6_Matplotlib
[ ]	13	SQLite: creating and connecting to an SQLite database	T4_SQLite
[ ]	14	SQLite: storing a Pandas DataFrame in a database table	T4_SQLite, T5_Pandas_NumPy
[ ]	15	SQLite: writing SELECT, WHERE, ORDER BY queries	T4_SQLite
[ ]	16	SQLite: writing UPDATE queries	T4_SQLite
[ ]	17	SQLite: using parameterised queries with ?	T4_SQLite
[ ]	18	SQLite: committing database changes	T4_SQLite
[ ]	19	SQLite: retrieving query results and exporting them to CSV	T4_SQLite, T3_Files_CSV_Exceptions
[ ]	20	PyTorch: setting a random seed before generating data	T7_PyTorch_Basics
[ ]	21	PyTorch: defining a simple neural network using torch.nn.Module	T7_PyTorch_Basics
[ ]	22	PyTorch: using Linear layers and ReLU	T7_PyTorch_Basics
[ ]	23	PyTorch: generating synthetic data (y = X @ w + b + noise)	T7_PyTorch_Basics
[ ]	24	PyTorch: understanding the shapes of X and y	T7_PyTorch_Basics
[ ]	25	PyTorch: choosing the required loss function and optimiser	T7_PyTorch_Basics
[ ]	26	PyTorch: writing a basic training loop	T7_PyTorch_Basics
[ ]	27	PyTorch: recording and plotting the training loss	T7_PyTorch_Basics, T6_Matplotlib

(The checklist has 22 bullets. "Identifying and removing duplicates" is split into 2 cards, "scatter / line / bar / box" into 4 cards, and "SELECT, WHERE, ORDER BY and UPDATE" into 2 cards, giving 27.)

PANDAS (cards 1-7)

Card 1 - Load a CSV file

Checklist: "loading CSV files"

Analogy: Opening a spreadsheet file and pouring it into a table (DataFrame) that Python can work on.

```
df = pd.read_csv("sales.csv")
print(df.head())
print(df.dtypes)
```

- `pd.read_csv("sales.csv")` reads the file (path in quotes) and RETURNS a DataFrame - you must store it in `df`.
- `df.head()` shows the first 5 rows; `df.dtypes` shows each column's type.

Check it worked: `print(df.head())` shows the header names, and `df.dtypes` shows `Quantity int64`, `Price float64` (numbers must not be `object`).

Common slip: wrong name or wrong folder gives `FileNotFoundError: [Errno 2] No such file or directory: 'sale.csv'`.

Self-test: What type of object does `read_csv` return, and what does `df.head(3)` show?

Card 2 - Check the shape

Checklist: "checking the shape of a DataFrame"

Analogy: Counting the rows and columns of the table before you trust it.

```
print(df.shape)                   # (6, 4)  -> (rows, columns)
rows, cols = df.shape
print(rows, cols, len(df))
```

- `df.shape` is a TUPLE `(rows, columns)` - an attribute, no brackets after it.
- `rows, cols = df.shape` unpacks the tuple; `len(df)` also gives the row count.

Check it worked: `print(df.shape)` prints `(6, 4)`: 6 data rows (the header is not counted) and 4 columns.

Common slip: `df.shape()` gives `TypeError: 'tuple' object is not callable`.

Self-test: A file has a header and 150 lines of data and 5 columns. What does `df.shape` print, and which index is the row count?

Card 3 - Identify duplicates

Checklist: "identifying and removing duplicates" (identify)

Analogy: A teacher scanning the register for a name written twice, without crossing anything out yet.

```
print(df.duplicated().sum())                      # 1  exact repeat rows
print(df[df.duplicated(keep=False)])              # show every copy
print(df.duplicated(subset=["Product"]).sum())    # 2  repeated Products
```

- `df.duplicated()` gives True/False per row: True for the 2nd, 3rd... copy (the first copy is False).
- `.sum()` counts the Trues (True = 1).
- `keep=False` marks ALL copies, so you can see the pair together.
- `subset=["Product"]` compares only that column instead of the whole row.

Check it worked: `df.duplicated().sum()` prints `1`, and the `keep=False` line prints the two identical Book rows.

Common slip: forgetting the brackets: `df.duplicated.sum()` gives `AttributeError: 'function' object has no attribute 'sum'`.

Self-test: Which copy does `duplicated()` leave as False by default, and which argument flags all copies?

Card 4 - Remove duplicates

Checklist: "identifying and removing duplicates" (remove)

Analogy: Crossing out the repeat names, keeping the first one of each.

```
clean = df.drop_duplicates()                       # whole row must match
by_key = df.drop_duplicates(subset=["Product"], keep="first")
clean = clean.reset_index(drop=True)
print(df.shape, clean.shape, by_key.shape)         # (6, 4) (5, 4) (4, 4)
```

- `df.drop_duplicates()` RETURNS a new DataFrame without repeat rows; `df` itself is unchanged.
- `subset=["Product"], keep="first"` keeps the first row for each Product.
- `reset_index(drop=True)` renumbers 0,1,2... (the removal leaves gaps).

Check it worked: the printed shapes are `(6, 4) (5, 4) (4, 4)` and `clean.duplicated().sum()` is `0`.

Common slip: calling `df.drop_duplicates()` without `clean = ...` - no error, but `df.shape` is still `(6, 4)`.

Self-test: Write the one line that keeps only the first row for each (Date, Product) pair and stores the result in `clean`.

Card 5 - Merge on a common column

Checklist: "merging DataFrames using a common column"

Analogy: Stapling each sale to its product description by matching the Product name (a lookup).

```
sales = pd.read_csv("sales.csv")
products = pd.read_csv("products.csv")
inner = sales.merge(products, on="Product", how="inner")
left = pd.merge(sales, products, on="Product", how="left", indicator=True)
print(sales.shape, products.shape, inner.shape, left.shape)
print(left["_merge"].value_counts())
```

- `on="Product"` is the column that exists in BOTH tables (spelled identically).
- `how="inner"` keeps only matching rows; `how="left"` keeps ALL left rows (no match gives NaN).
- `indicator=True` adds a `_merge` column: `both` / `left_only` / `right_only`.
- `pd.merge(a, b, ...)` and `a.merge(b, ...)` do the same thing.

Check it worked: the shapes print `(6, 4) (4, 2) (5, 5) (6, 6)`, and `_merge` shows `left_only` once (Cap).

Common slip: `on="product"` gives `KeyError: 'product'` (names are case-sensitive).

Self-test: `sales` has 6 rows and one Product is missing from `products`. How many rows after `inner`, after `left`, and which `_merge` value marks the missing one?

Card 6 - Select and filter

Checklist: "selecting and filtering data"

Analogy: Choosing columns = choosing which columns to look at; filtering = a sieve that keeps only rows where the condition is True.

```
two = df[["Product", "Price"]]                                    # 2 columns
big = df[df["Quantity"] > 2]                                      # one condition
both = df[(df["Product"] == "Pen") & (df["Price"] > 1)]           # AND
either = df[(df["Product"] == "Bag") | (df["Quantity"] >= 3)]     # OR
sub = df.loc[df["Price"] > 10, ["Product", "Price"]]              # rows + columns
pick = df[df["Product"].isin(["Pen", "Cap"])]                     # in a list
print(two.shape, big.shape, both.shape, either.shape, sub.shape, pick.shape)
```

- `df[["Product", "Price"]]` - a LIST inside `[]` selects columns; `df["Price"]` (no list) is one Series.
- `df["Quantity"] > 2` makes a True/False column; `df[ that ]` keeps the True rows.
- `&` = and, `|` = or. Each condition needs its own `( )`.
- `df.loc[rows_condition, [columns]]` filters rows and picks columns in one go.

Check it worked: the shapes print `(6, 2) (3, 4) (2, 4) (4, 4) (3, 2) (3, 4)`.

Common slip: using `and` gives `ValueError: The truth value of a Series is ambiguous. Use a.empty, a.item(), a.any() or a.all().`; missing brackets gives `TypeError: Cannot perform 'rand_' with a dtyped [float64] array and scalar of type [bool]`.

Self-test: Write the filter for rows where Product is "Pen" OR Quantity is at least 3.

Card 7 - Save to CSV

Checklist: "saving a DataFrame to a CSV file"

Analogy: Saving the finished table back to a spreadsheet file - without the row numbers.

```
clean.to_csv("clean_sales.csv", index=False)
back = pd.read_csv("clean_sales.csv")
print(back.shape)                                  # (5, 4)
```

- `clean.to_csv("clean_sales.csv", ...)` writes the file in the current folder.
- `index=False` stops pandas writing the 0,1,2... row numbers as an extra column.
- `read_csv` straight afterwards is the proof that the file is good.

Check it worked: `print(back.shape)` prints `(5, 4)` and `"Unnamed: 0"` is NOT in `back.columns`.

Common slip: forgetting `index=False` - no error, but reading the file back shows an extra column `Unnamed: 0`.

Self-test: What does `index=False` do, and in which folder does the file appear if you give no path?

MATPLOTLIB (cards 8-12)

Each chart card does the same 4 steps: draw, `title`, `xlabel`, `ylabel`, then `plt.show()` LAST. Run the check line BEFORE `plt.show()` (after `show` the figure is closed).

Card 8 - Scatter plot

Checklist: "creating simple plots such as scatter plots..." and "adding appropriate titles and axis labels"

Analogy: Sticking one dot on graph paper for every (x, y) pair of measurements.

```
hours = [1, 2, 3, 4, 5, 6]
marks = [48, 55, 61, 70, 74, 85]
plt.scatter(hours, marks)
plt.title("Study time vs marks")
plt.xlabel("Hours studied")
plt.ylabel("Mark (%)")
plt.show()
```

- `plt.scatter(hours, marks)` - x list first, y list second, SAME length; dots only, no line.
- `plt.title / plt.xlabel / plt.ylabel` write the three captions (strings).
- `plt.show()` opens the window and waits until you close it.

Check it worked: before `show`, `ax = plt.gca(); print(ax.get_title(), ax.get_xlabel(), ax.get_ylabel())` prints your three captions; the window shows 6 separate dots.

Common slip: lists of different length give `ValueError: x and y must be the same size`.

Self-test: When is a scatter plot the right chart rather than a line plot?

Card 9 - Line plot

Checklist: "creating simple plots such as ... line plots..." and "adding appropriate titles and axis labels"

Analogy: Connecting the dots in time order, like a temperature chart.

```
months = ["Jan", "Feb", "Mar", "Apr"]
total = [120, 150, 90, 180]
plt.plot(months, total, marker="o")
plt.title("Monthly sales")
plt.xlabel("Month")
plt.ylabel("Total sales")
plt.show()
```

- `plt.plot(x, y)` joins the points in the order given - so sort the x values (months in order) first.
- `marker="o"` also draws a dot at each point.
- The 3 caption calls and `plt.show()` are identical to card 8.

Check it worked: `print(plt.gca().get_title())` prints `Monthly sales` before `show`; the window shows ONE connected line with 4 dots.

Common slip: `ValueError: x and y must have same first dimension, but have shapes (3,) and (2,)`.

Self-test: Which kind of data belongs on the x-axis of a line plot?

Card 10 - Bar chart

Checklist: "creating simple plots such as ... bar charts..." and "adding appropriate titles and axis labels"

Analogy: One bar per category, as tall as its total.

```
df = pd.read_csv("sales.csv")
totals = df.groupby("Product")["Quantity"].sum()     # index = Product
plt.bar(totals.index, totals.values)
plt.title("Quantity sold per product")
plt.xlabel("Product")
```

```
plt.ylabel("Total quantity")
plt.show()
```

- `df.groupby("Product")["Quantity"].sum()` adds up Quantity per Product (gives Bag 1, Book 4, Cap 3, Pen 15).
- `totals.index` = the category names (x), `totals.values` = the heights (y).
- Use the grouped result for BOTH x and y so the lengths match.

Check it worked: `print(totals)` shows 4 products; before `show`, `len(plt.gca().patches)` is `4` (one rectangle per bar).

Common slip: `plt.bar(df["Product"], totals)` (6 x-values, 4 heights) gives `ValueError: shape mismatch: objects cannot be broadcast to a single shape. Mismatch is between 'x' with shape (6,) and 'height' with shape (4,).`

Self-test: Which pandas line turns 6 sales rows into one total per product, ready for `plt.bar`?

Card 11 - Box plot

Checklist: "creating simple plots such as ... box plots" and "adding appropriate titles and axis labels"

Analogy: A summary "sandwich" of each group: the box holds the middle half of the values, the line inside is the median, dots far away are outliers.

```
group_a = [55, 60, 62, 65, 70, 98]
group_b = [40, 52, 58, 61, 63, 66]
plt.boxplot([group_a, group_b])                # one box per list
plt.xticks([1, 2], ["A", "B"])                 # name the boxes (works on every version)
plt.title("Marks by group")
plt.xlabel("Group")
plt.ylabel("Mark (%)")
plt.show()
```

- `plt.boxplot([group_a, group_b])` - a LIST of lists gives one box per group; one list gives one box.
- `plt.xticks([1, 2], ["A", "B"])` names the boxes (box 1 sits at x=1, box 2 at x=2). **This works on every matplotlib version.** The keyword `plt.boxplot(..., tick_labels=[...])` only exists in matplotlib 3.9+, and the old `labels=` was removed in 3.11, so avoid both.
- From a DataFrame: `plt.boxplot(df["Price"])` or `df.boxplot(column="Price")`.

Check it worked: before `show`, `[t.get_text() for t in plt.gca().get_xticklabels()]` prints `['A', 'B']`; the window shows 2 boxes (the 98 is a lone dot).

Common slip: `plt.boxplot(..., labels=["A", "B"])` gives `TypeError: boxplot() got an unexpected keyword argument 'labels'. Did you mean 'label'?` on matplotlib 3.11.

Self-test: What do the box, the line inside it, the whiskers and the lone dots show?

Card 12 - Titles and axis labels

Checklist: "adding appropriate titles and axis labels"

Analogy: The caption under a photo: a chart without title and labelled axes is a photo nobody can read.

```
plt.plot([1, 2, 3], [2, 4, 8], label="trial 1")
plt.title("Growth")
plt.xlabel("Step")
plt.ylabel("Value")
plt.legend()
fig, ax = plt.subplots()                       # object style, same result
ax.plot([1, 2, 3], [2, 4, 8])
ax.set_title("Growth")
ax.set_xlabel("Step")
ax.set_ylabel("Value")
plt.show()
```

- `plt.title / xlabel / ylabel` act on the current chart; `label="trial 1"` + `plt.legend()` adds a key.
- With `fig, ax = plt.subplots()` the same jobs are methods of `ax`: `set_title`, `set_xlabel`, `set_ylabel`.

- Labels should say WHAT and the UNIT: "Height (cm)", not "x".

Check it worked: `print(ax.get_title(), ax.get_xlabel(), ax.get_ylabel())` prints `Growth Step Value`.

Common slip: mixing the styles: `plt.set_title("x")` gives `AttributeError: module 'matplotlib.pyplot' has no attribute 'set_title'` (it is `plt.title` or `ax.set_title`).

Self-test: Give the three `plt` calls and the three `ax` calls for title, x label and y label.

SQLITE WITH PYTHON (cards 13-19)

Card 13 - Create and connect to a database

Checklist: "creating and connecting to an SQLite database"

Analogy: Opening a filing cabinet (creating it if missing) and hiring a clerk (the cursor) who carries your notes.

```
conn = sqlite3.connect("shop.db")      # creates the file if missing
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS sales "
            "(Product TEXT, Quantity INTEGER, Price REAL)")
conn.commit()
print(cur.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall())
conn.close()
```

- `sqlite3.connect("shop.db")` opens (or creates) the file and returns the connection.
- `conn.cursor()` is the clerk; `cur.execute(sql)` hands over one SQL note.
- `IF NOT EXISTS` makes the script safe to run twice. `TEXT / INTEGER / REAL` are the column types.
- `conn.close()` locks the cabinet (needs `import sqlite3` at the top).

Check it worked: the `sqlite_master` line prints `[('sales',)]`, and `shop.db` now exists in the folder.

Common slip: running `CREATE TABLE sales ...` twice gives `sqlite3.OperationalError: table sales already exists` (add `IF NOT EXISTS`).

Self-test: What does `sqlite3.connect("shop.db")` do when `shop.db` does not exist, and what is the cursor for?

Card 14 - Store a DataFrame in a table

Checklist: "storing a Pandas DataFrame in a database table"

Analogy: Tipping the whole spreadsheet into a drawer of the cabinet in one move.

```
df = pd.read_csv("sales.csv")
conn = sqlite3.connect("shop.db")
df.to_sql("sales", conn, index=False, if_exists="replace")
print(pd.read_sql_query("SELECT COUNT(*) AS n FROM sales", conn))
conn.close()
```

- `df.to_sql("sales", conn, ...)` = table name, then the open connection; it creates the table for you and commits.
- `index=False` stops the row-number column being stored.
- `if_exists`: `"fail"` (default, error), `"replace"` (drop and rebuild), `"append"` (add rows to the existing table).
- `pd.read_sql_query(sql, conn)` runs a SELECT and returns a DataFrame.

Check it worked: the COUNT query prints `n = 6`; `PRAGMA table_info(sales)` lists Date, Product, Quantity, Price and no `index` column.

Common slip: running it twice without `if_exists` gives `ValueError: Table 'sales' already exists.`

Self-test: Which `if_exists` value would you use to add new rows to a table that already has data?

Card 15 - SELECT, WHERE, ORDER BY

Checklist: "writing SELECT, WHERE, ORDER BY ... queries"

Analogy: Asking the clerk: "From the drawer, fetch these columns, only the cards matching this rule, sorted this way."

```
cur.execute("SELECT Product, Quantity FROM sales "
            "WHERE Quantity >= 3 ORDER BY Quantity DESC")
rows = cur.fetchall()
print(rows)                       # [('Pen', 10), ('Pen', 5), ('Cap', 3)]
```

- `SELECT Product, Quantity FROM sales` = which columns, which table (`*` = all columns).

- `WHERE Quantity >= 3` keeps matching rows; combine with `AND` / `OR`; text values use quotes: `Product = 'Pen'`.
- `ORDER BY Quantity DESC` sorts biggest first (`ASC` = smallest first, the default).
- The clause ORDER is fixed: SELECT, FROM, WHERE, ORDER BY. `cur.fetchall()` returns a list of tuples.

Check it worked: `print(rows)` prints `[('Pen', 10), ('Pen', 5), ('Cap', 3)]` - 3 tuples, biggest first.

Common slip: `ORDER BY` before `WHERE` gives `sqlite3.OperationalError: near "WHERE": syntax error`; a wrong column gives `no such column: Qty`.

Self-test: Write a query for Product and Price of rows with Price over 5, cheapest first.

Card 16 - UPDATE

Checklist: "writing ... UPDATE queries"

Analogy: Finding the cards that match and rewriting one box on each of them.

```
cur.execute("UPDATE sales SET Price = ? WHERE Product = ?", (2.0, "Pen"))
print(cur.rowcount)              # 2 rows changed
conn.commit()
```

- `UPDATE sales SET Price = ?` names the table and the column to change.
- `WHERE Product = ?` picks WHICH rows. The `?` values come from the tuple, in order: `(2.0, "Pen")`.
- `cur.rowcount` = how many rows the UPDATE changed (0 means nothing matched - not an error).
- `conn.commit()` saves it.

Check it worked: `print(cur.rowcount)` prints `2`, and `SELECT Price FROM sales WHERE Product = 'Pen'` returns `[(2.0,), (2.0,)]`.

Common slip: forgetting `WHERE` - no error, but `cur.rowcount` is `6` and EVERY price changed.

Self-test: What does `cur.rowcount` print if no Product equals "Nothing"?

Card 17 - Parameterised queries with ?

Checklist: "using parameterised queries with ?"

Analogy: A form with blanks: the question is fixed, the values are slotted in separately and safely.

```
name = "Pen"
min_qty = 6
cur.execute("SELECT Product, Quantity FROM sales "
            "WHERE Product = ? AND Quantity >= ?", (name, min_qty))   # tuple
print(cur.fetchall())            # [('Pen', 10)]
cur.execute("SELECT * FROM sales WHERE Product = ?", (name,))        # 1 value
print(len(cur.fetchall()))       # 2
```

- One `?` per value; no quotes around `?`, even for text.
- The second argument is a TUPLE in the same order as the `?`s.
- ONE value still needs a tuple, so `(name,)` WITH the comma.
- Never build the SQL with an f-string: `?` stops user text from changing the query.

Check it worked: the first print gives `[('Pen', 10)]` and the second gives `2`.

Common slip: `cur.execute("... WHERE Product = ?", "Pen")` gives `sqlite3.ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 3 supplied.` (a bare string counts as 3 letters).

Self-test: Why does a single value need `(name,)` and not `(name)`?

Card 18 - Commit

Checklist: "committing database changes"

Analogy: Pressing Save. Until then your change sits in a draft that only you can see.

```
cur.execute("UPDATE sales SET Quantity = ? WHERE Product = ?", (99, "Bag"))
other = sqlite3.connect("shop.db")                    # a second "viewer"
print(other.execute("SELECT Quantity FROM sales WHERE Product='Bag'").fetchone())  # (1,)
conn.commit()                                         # now it is saved
print(other.execute("SELECT Quantity FROM sales WHERE Product='Bag'").fetchone())  # (99,)
conn.close()
other.close()
```

- `conn.commit()` is called on the CONNECTION (not the cursor) after INSERT / UPDATE / DELETE.
- Before the commit, a second connection still sees the old value `(1,)`; after it, `(99,)`.
- Closing without commit throws the change away. SELECT needs no commit. (`df.to_sql` commits for you.)

Check it worked: the two prints show `(1,)` then `(99,)`; a fresh `connect` after `close()` also shows `(99,)`.

Common slip: no error at all - you forget `conn.commit()`, close, and the UPDATE is silently lost.

Self-test: Which object has `commit()`, and when must you call it?

Card 19 - Retrieve results and export to CSV

Checklist: "retrieving query results and exporting them to CSV"

Analogy: The clerk hands you a pile of cards (rows) plus the column headings; you copy them onto a spreadsheet.

```
cur.execute("SELECT Product, Quantity FROM sales ORDER BY Quantity DESC")
rows = cur.fetchall()                              # list of tuples
cols = [d[0] for d in cur.description]             # column names
pd.DataFrame(rows, columns=cols).to_csv("result.csv", index=False)
pd.read_sql_query("SELECT * FROM sales", conn).to_csv("all.csv", index=False)
with open("result2.csv", "w", newline="") as f:    # csv module version
    w = csv.writer(f)
    w.writerow(cols)
    w.writerows(rows)
```

- `cur.fetchall()` = all rows (list of tuples); `fetchone()` = one tuple.
- `cur.description` holds one tuple per column; item `[0]` is the column name (`import csv` needed for the last part).
- Pandas route: `DataFrame(rows, columns=cols).to_csv(..., index=False)`, or the one-liner `pd.read_sql_query(sql, conn).to_csv(...)`.
- csv route: `writerow(cols)` writes the header, `writerows(rows)` writes all rows.

Check it worked: `pd.read_csv("result.csv").shape` prints `(6, 2)` and `all.csv` gives `(6, 4)`.

Common slip: calling `fetchall()` a second time returns `[]` with no error - the cursor is already used up.

Self-test: After `fetchall()`, where do you get the column names for the CSV header?

PYTORCH (cards 20-27)

All torch cards start with `import torch` and `import torch.nn as nn`.

Card 20 - Set a random seed

Checklist: "Setting a random seed before generating data so that your results are reproducible each time you run your program."

Analogy: Shuffling a deck the same way every time, so a re-run deals the same cards.

```
import torch
torch.manual_seed(42)
a = torch.randn(3)
torch.manual_seed(42)              # same seed again ...
b = torch.randn(3)
print(torch.equal(a, b))           # True -> identical numbers
```

- `torch.manual_seed(42)` fixes torch's random generator (any integer works; use the same one every run).
- It must come BEFORE the first `torch.randn` / `Linear(...)` call that uses randomness.
- If you use NumPy random numbers too, also call `np.random.seed(42)`.

Check it worked: `print(torch.equal(a, b))` prints `True`; re-run the whole script and the printed numbers are identical.

Common slip: seeding AFTER generating the data - no error, but the numbers change on every run.

Self-test: Why must the seed line come before the data is generated?

Card 21 - Define a network with nn.Module

Checklist: "defining a simple neural network using torch.nn.Module"

Analogy: A recipe card: `__init__` lists the ingredients (layers), `forward` says in which order to use them.

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()                 # MUST be first
        self.fc1 = nn.Linear(3, 8)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(8, 1)

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

model = Net()
print(model(torch.randn(5, 3)).shape)      # torch.Size([5, 1])
```

- `class Net(nn.Module)` - inherit from `nn.Module`.
- `super().__init__()` sets up the Module machinery; it must run before any `self.xxx = layer`.
- `forward(self, x)` is the data path: layer 1, ReLU, layer 2. You call `model(x)`, never `model.forward(x)` directly.
- `model.parameters()` automatically finds the layers' weights (here 4 tensors).

Check it worked: `print(model(torch.randn(5, 3)).shape)` prints `torch.Size([5, 1])` - 5 samples in, 5 predictions out.

Common slip: no `super().__init__()` gives `AttributeError: cannot assign module before Module.__init__() call`.

Self-test: Which two methods does every `nn.Module` class you write need, and what does each do?

Card 22 - Linear layers and ReLU

Checklist: "using Linear layers and ReLU"

Analogy: `Linear` = a mixing desk that turns 3 input numbers into 8 weighted sums; `ReLU` = a gate that lets positives through and turns negatives into 0.

```
lin = nn.Linear(3, 8)                       # 3 numbers in, 8 numbers out
x = torch.tensor([[1.0, -2.0, 0.5]])        # shape (1, 3)
h = lin(x)                                  # shape (1, 8)
r = nn.ReLU()(h)                            # negatives -> 0
print(lin.weight.shape, lin.bias.shape, h.shape)
print((r >= 0).all())                       # tensor(True)
```

- `nn.Linear(in_features, out_features)` computes `x @ W.T + b`; `in_features` must equal the last size of the input.
- `lin.weight` has shape `(out, in)` = `(8, 3)`; `lin.bias` has shape `(8,)`.
- `nn.ReLU()` has no weights: `max(0, value)` for every number.
- Stacking: `nn.Sequential(nn.Linear(3, 8), nn.ReLU(), nn.Linear(8, 1))`. The LAST layer has no ReLU for a continuous target.

Check it worked: the print shows `torch.Size([8, 3]) torch.Size([8]) torch.Size([1, 8])`, then `tensor(True)`.

Common slip: wrong `in_features`: `nn.Linear(4, 8)(torch.randn(1, 3))` gives `RuntimeError: mat1 and mat2 shapes cannot be multiplied (1x3 and 4x8)`.

Self-test: What shape is the weight matrix of `nn.Linear(3, 8)`, and what does ReLU do to -2.0?

Card 23 - Generate synthetic data

Checklist: "generating synthetic data (Generate a continuous target: y = X @ w + b + noise)"

Analogy: Inventing a "true" rule, then adding a little static so the network has something to discover.

```
torch.manual_seed(42)                       # seed BEFORE generating
n, d = 100, 3
X = torch.randn(n, d)                       # (100, 3)
w = torch.tensor([[2.0], [-1.0], [0.5]])    # (3, 1)  column!
b = 1.0
noise = 0.1 * torch.randn(n, 1)             # (100, 1)
y = X @ w + b + noise                       # (100, 1)
print(X.shape, w.shape, y.shape)
```

- `torch.randn(n, d)` = n rows (samples) x d columns (features) of random numbers.
- `w` is a COLUMN `(3, 1)` so that `X @ w` (matrix product) is `(100, 3) @ (3, 1) = (100, 1)`.
- `b` (a single number) is added to every row; `0.1 * torch.randn(n, 1)` is small random noise of the same shape as `y`.

Check it worked: `print(X.shape, w.shape, y.shape)` prints `torch.Size([100, 3]) torch.Size([3, 1]) torch.Size([100, 1])`.

Common slip: a flat `w = torch.tensor([2., -1., 0.5])` gives `X @ w` shape `(100,)`; adding `noise (100, 1)` silently broadcasts `y` to `(100, 100)` with no error.

Self-test: What shapes must X, w, noise and y have for 100 samples and 3 features?

Card 24 - Understand the shapes of X and y

Checklist: "understanding the shapes of X and y"

Analogy: A spreadsheet: X is the table of inputs (one row per sample), y is the single answer column beside it.

```
print(X.shape, y.shape)                     # [100, 3] [100, 1]
n_samples, n_features = X.shape
pred = model(X)
print(pred.shape == y.shape)                # True -> safe for MSELoss
y_flat = y.squeeze()                        # (100,)  drop the size-1 axis
y_col = y_flat.reshape(-1, 1)               # (100, 1) back to a column
```

- `X.shape` is `(samples, features)`: 100 rows, 3 features. The first layer is `nn.Linear(n_features, ...)`.

- `y` is `(samples, 1)`: one target number per sample; the model output must have the SAME shape.
- `.squeeze()` removes size-1 axes; `.reshape(-1, 1)` makes a column (`-1` = "work it out").

Check it worked: `print(pred.shape == y.shape)` prints `True`.

Common slip: `nn.MSELoss()(pred, y_flat)` with `(100, 1)` vs `(100,)` gives a `UserWarning: Using a target size (torch.Size([100])) that is different to the input size (torch.Size([100, 1])). This will likely lead to incorrect results due to broadcasting.` - and a wrong loss.

Self-test: X is `(100, 3)`. What is `n_features`, what shape must `pred` and `y` have, and what does `-1` mean in `reshape(-1, 1)`?

Card 25 - Choose the loss function and optimiser

Checklist: "choosing the required loss function and optimiser"

Analogy: The loss is the scoreboard (how wrong are we?); the optimiser is the coach who adjusts the weights to lower the score.

```
model = nn.Linear(3, 1)
loss_fn = nn.MSELoss()                    # continuous target -> MSE
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
# optimizer = torch.optim.Adam(model.parameters(), lr=0.05)   # alternative
loss = loss_fn(model(X), y)
print(loss.item())
```

- `nn.MSELoss()` = mean squared error: the right choice for a CONTINUOUS target (classes would use `CrossEntropyLoss`).
- `torch.optim.SGD(model.parameters(), lr=0.05)` - first argument = the weights to adjust, `lr` = step size.
- `Adam(...)` has the same call and usually copes with the learning rate better.
- `loss_fn(prediction, target)` returns a one-number tensor; `.item()` turns it into a plain float.

Check it worked: `print(loss.item())` prints one positive float (around 8 for the untrained model on this data).

Common slip: passing the model, not its parameters: `torch.optim.SGD(model, lr=0.05)` gives `TypeError: 'Linear' object is not iterable`.

Self-test: Which loss for a continuous target, and what is the first argument of every optimiser?

Card 26 - Write a basic training loop

Checklist: "writing a basic training loop"

Analogy: Practice rounds: guess, check the score, learn from the mistake, adjust, repeat.

```
for epoch in range(100):
    pred = model(X)                       # 1 forward pass
    loss = loss_fn(pred, y)               # 2 how wrong?
    optimizer.zero_grad()                 # 3 clear old gradients
    loss.backward()                       # 4 new gradients
    optimizer.step()                      # 5 nudge the weights
print(loss.item())
```

- `for epoch in range(100)` = 100 passes over the data.
- Steps 1-5 ALWAYS in this order: forward, loss, `zero_grad`, `backward`, `step`.
- `zero_grad()` is needed because PyTorch ADDS to old gradients. `backward()` computes them; `step()` uses them.

Check it worked: print the loss before the loop and after it: it drops from roughly 8 to below 0.1.

Common slip: computing `pred = model(X)` once OUTSIDE the loop gives `RuntimeError: Trying to backward through the graph a second time (or directly access saved tensors after they have already been freed).`

Self-test: Put in order: `optimizer.step()`, `loss.backward()`, forward pass, `optimizer.zero_grad()`, `loss = loss_fn(...)`.

Card 27 - Record and plot the training loss

Checklist: "recording and plotting the training loss"

Analogy: Writing the score of every practice round in a notebook, then drawing the notebook as a graph.

```
losses = []
for epoch in range(100):
    loss = loss_fn(model(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())            # .item() -> plain float
plt.plot(losses)
plt.title("Training loss")
plt.xlabel("Epoch")
plt.ylabel("MSE loss")
plt.show()
```

- `losses = []` BEFORE the loop; `losses.append(loss.item())` INSIDE it, once per epoch.
- `.item()` converts the 1-number tensor to a float that matplotlib can plot.
- `plt.plot(losses)` uses the list position (0, 1, 2...) as the x-axis = epoch. Same caption calls as card 12.

Check it worked: `print(len(losses), losses[0], losses[-1])` prints `100`, a big number, then a much smaller one; the curve falls.

Common slip: `losses.append(loss)` (no `.item()`) gives `RuntimeError: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead.` at `plt.plot`.

Self-test: Why `loss.item()` and not just `loss`, and what should a healthy loss curve look like?

Self-test answer key

Check yourself only AFTER you have answered aloud.

- **1** - A DataFrame; `df.head(3)` shows the first 3 rows.
- **2** - `(150, 5)`; `df.shape[0]` is the row count.
- **3** - The first copy is False; `keep=False` flags all copies.
- **4** - `clean = df.drop_duplicates(subset=["Date", "Product"], keep="first")`
- **5** - `inner` 5 rows, `left` 6 rows; the missing one is `left_only`.
- **6** - `df[(df["Product"] == "Pen") | (df["Quantity"] >= 3)]` (each condition in brackets).
- **7** - It leaves out the row numbers; the file appears in the current working folder.
- **8** - When both x and y are measurements and you want to see the relationship (no ordering, dots not joined).
- **9** - Ordered data, usually time (days, months, epochs).
- **10** - `df.groupby("Product")["Quantity"].sum()`, then `plt.bar(s.index, s.values)`.
- **11** - Box = middle 50% (Q1 to Q3), line = median, whiskers = the normal range, lone dots = outliers.
- **12** - `plt.title / plt.xlabel / plt.ylabel`; `ax.set_title / ax.set_xlabel / ax.set_ylabel`.
- **13** - It creates the file; the cursor sends SQL (`execute`) and fetches results.
- **14** - `if_exists="append"`.
- **15** - `SELECT Product, Price FROM sales WHERE Price > 5 ORDER BY Price ASC`
- **16** - `0` - nothing matched, and no error.
- **17** - `(name,)` is a one-item tuple; `(name)` is just a string in brackets.
- **18** - The connection (`conn.commit()`), after every INSERT / UPDATE / DELETE.
- **19** - `[d[0] for d in cur.description]`
- **20** - Random numbers are drawn at creation; the seed must be set before so the sequence starts identically.
- **21** - `__init__` (create the layers, after `super().__init__()`) and `forward` (how data flows through them).
- **22** - `(8, 3)`; ReLU turns -2.0 into 0.0.
- **23** - X `(100, 3)`, w `(3, 1)`, noise `(100, 1)`, y `(100, 1)`.
- **24** - `n_features` = 3; both `(100, 1)`; `-1` = "work out this length yourself".
- **25** - `nn.MSELoss()`; `model.parameters()`.
- **26** - Forward pass, `loss = loss_fn(...)`, `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`.
- **27** - `.item()` gives a plain float (a tensor with gradients cannot be plotted); the curve should fall steeply, then flatten.

Quick reference box

```
PANDAS   read_csv(f) | df.shape | df.duplicated().sum() | df.drop_duplicates(subset=[..])
         a.merge(b, on="col", how="inner/left", indicator=True) | df[(c1) & (c2)] | df.to_csv(f,
index=False)
PLOTS    scatter(x,y) | plot(x,y) | bar(names,heights) | boxplot([l1,l2]); xticks([1,2],[..])
         then title() xlabel() ylabel() show()          (boxplot labels= was removed in matplotlib 3.11)
SQLITE   connect -> cursor -> execute(sql, (values,)) -> commit -> fetchall -> close
         df.to_sql("t", conn, index=False, if_exists="replace") | pd.read_sql_query(sql, conn)
         SELECT cols FROM t WHERE cond ORDER BY col DESC | UPDATE t SET col=? WHERE col=? | cur.rowcount
TORCH    manual_seed(42) FIRST | y = X @ w + b + noise   X(n,d) w(d,1) y(n,1)
         class Net(nn.Module): super().__init__(); Linear -> ReLU -> Linear; forward()
         MSELoss + SGD/Adam(model.parameters(), lr) | pred, loss, zero_grad, backward, step |
losses.append(loss.item())
```